**МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ**
**ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ**
**РАДІОЕЛЕКТРОНІКИ**

Кафедра системотехніки

# ЗВІТ

з лабораторної роботи №1
з дисципліни «Технології високопродуктивних обчислень»
на тему: «Багатопоточне програмування»
Варіант 9

Виконав:
ст. гр. КНТ-23-1
Кравченко Р.

Перевірив:
проф. каф. СТ
Міщеряков Ю.В.

Харків 2026

# 1 МЕТА РОБОТИ

Вивчити способи організації паралельних багатопоточних додатків на прикладі розробки програмного засобу обробки зображення – накладання фільтра.

# 2 ЗАВДАННЯ

Варіант 9 – шум Перліна (Perlin Noise). Розробити програмний засіб, який накладає на зображення шум Перліна, розподіляючи обробку між кількома потоками. Розрахувати теоретичні прискорення та ефективність розробленого паралельного алгоритму, зібрати статистику витрат часу на обробку зображення в одному та декількох потоках (1, 2, 4, 6, 8, 10, 12, 14, 16), побудувати графік залежності часу обробки від кількості потоків та порівняти отримані практичні результати з теоретичними.

# 3 ХІД ВИКОНАННЯ РОБОТИ

## 3.1 Алгоритм шуму Перліна

Шум Перліна – це градієнтний шум, який дає плавну псевдовипадкову функцію $n(x, y)$ зі значеннями приблизно в межах $[-1; 1]$. У роботі використано покращену версію алгоритму К. Перліна (2002 р.). Площина розбивається на квадратні клітинки цілочисельної решітки; кожному вузлу решітки за таблицею перестановок із 256 елементів зіставляється псевдовипадковий градієнт. Для точки $(x, y)$:

– визначаються клітинка, у яку потрапляє точка, та дробові координати всередині неї $x_f$, $y_f$;

– для кожного з чотирьох кутів обчислюється скалярний добуток градієнта на вектор від кута до точки;

– чотири значення інтерполюються з вагами, отриманими згладжувальною кривою

$$fade(t) = 6t^5 - 15t^4 + 10t^3,$$

що забезпечує неперервність першої та другої похідних на межах клітинок.

Щоб отримати природну «хмарну» текстуру, використовується фрактальна сума октав (fBm):

$$N(x, y) = \Sigma_{o=0}^{K-1} a^o \cdot n(2^o \cdot s \cdot x, 2^o \cdot s \cdot y) / \Sigma_{o=0}^{K-1} a^o,$$

де $K = 6$ – кількість октав, $a = 0{,}5$ – коефіцієнт згасання амплітуди (persistence), $s = 1/160$ – масштаб (розмір найбільшої «плями» ≈ 160 пікселів).

Накладання шуму на зображення: до кожного каналу (R, G, B) пікселя додається зсув яскравості, пропорційний значенню шуму, з обмеженням результату діапазоном $[0; 255]$:

$$B(x, y) = \text{clamp}(A(x, y) + \text{round}(k \cdot N(x, y)), 0, 255),$$

де $A$ – вхідне зображення, $B$ – результат, $k = 70$ – сила ефекту. Значення шуму для пікселя залежить лише від його координат, тому пікселі обробляються незалежно один від одного – задача має природний паралелізм за даними.

### 3.2 Паралельний алгоритм

Використано декомпозицію за даними: зображення висотою $H$ рядків ділиться на $p$ горизонтальних смуг майже однакової висоти, по одній смузі на потік. Потік $t$ ($t = 0, \ldots, p - 1$) обробляє рядки від $\lfloor H \cdot t/p \rfloor$ до $\lfloor H \cdot (t+1)/p \rfloor - 1$. Головний потік створює $p$ потоків (клас System.Threading.Thread), запускає їх і чекає на завершення кожного (Join).

Синхронізація під час обробки не потрібна: вхідне зображення та таблиця перестановок шуму лише читаються, а кожен потік записує результат тільки у свої рядки вихідного масиву, тобто критичних ресурсів немає. Правильність перевіряється програмно: результат для кожної кількості потоків побайтово порівнюється з результатом в одному потоці.

### 3.3 Теоретична оцінка прискорення та ефективності

Тестове зображення має розмір $W \times H = 3840 \times 2400 = 9\ 216\ 000$ пікселів. Обробка одного пікселя займає однаковий час $\tau$ (6 октав шуму + 3 додавання), тому час послідовного виконання

$$T_1 = W \cdot H \cdot \tau.$$

За виміряним $T_1 = 1081{,}4$ мс отримуємо $\tau \approx 117{,}3$ нс на піксель.

Для $p$ потоків час визначається найвищою смугою та накладними витратами $t_0(p)$ на створення і очікування потоків:

$$T_p = \lceil H/p \rceil \cdot W \cdot \tau + t_0(p).$$

Накладні витрати виміряно окремою програмою (створення, запуск і очікування порожніх потоків, медіана 51 повтору): $t_0(1) \approx 0{,}16$ мс, $t_0(6) \approx 0{,}76$ мс, $t_0(12) \approx 1{,}29$ мс, $t_0(16) \approx 1{,}65$ мс. Це 0,02–0,15 % від $T_1$, отже частка послідовного коду за законом Амдала $f \approx 0$, і на паракомп’ютері

$$S_p = T_1/T_p \approx p, \quad E_p = S_p/p \approx 1.$$

Нерівномірність смуг (наприклад, для $p = 7$: 342 або 343 рядки) змінює прискорення менш ніж на 0,3 %. Реальну межу задає обладнання: процесор AMD Ryzen 5 5600 має 6 фізичних ядер і 12 логічних процесорів (SMT).

Потоки, що працюють на двох логічних процесорах одного ядра, ділять його обчислювальні блоки, тому теоретична оцінка для цієї машини

$$S_p \approx \min(p, 6), \quad E_p = S_p/p.$$

SMT може дати додатково 10–30 % понад 6 для обчислювальних задач, а при $p > 12$ потоків більше, ніж логічних процесорів, тому подальшого прискорення не очікується. Теоретичні значення наведено в таблиці 3.1.

Таблиця 3.1 – Теоретичні прискорення та ефективність

| $p$ | $S_p$ (паракомп'ютер) | $E_p$ (паракомп'ютер) | $S_p = \min(p, 6)$ | $E_p = S_p/p$ |
|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 1,00 |
| 2 | 2 | 1 | 2 | 1,00 |
| 3 | 3 | 1 | 3 | 1,00 |
| 4 | 4 | 1 | 4 | 1,00 |
| 5 | 5 | 1 | 5 | 1,00 |
| 6 | 6 | 1 | 6 | 1,00 |
| 7 | 7 | 1 | 6 | 0,86 |
| 8 | 8 | 1 | 6 | 0,75 |
| 10 | 10 | 1 | 6 | 0,60 |
| 12 | 12 | 1 | 6 | 0,50 |
| 14 | 14 | 1 | 6 | 0,43 |
| 16 | 16 | 1 | 6 | 0,38 |

### 3.4 Програмна реалізація

Програму написано мовою C# (.NET 10, консольний застосунок). Для читання та запису зображень використано System.Drawing (Bitmap.LockBits – доступ до пікселів як до масиву байтів BGR). Структура проєкту:

– Perlin.cs – клас шуму Перліна: таблиця перестановок, функції *fade*, *lerp*, *grad*, Noise (одна октава) та Fractal (сума октав);

– Program.cs – читання зображення, функція ProcessRows (обробка діапазону рядків), функція RunWithThreads (поділ на смуги, створення, запуск та очікування потоків), вимірювання часу, перевірка результату, збереження результатів у results.csv та зображень.

Час вимірюється класом Stopwatch від створення потоків до завершення останнього з них; читання та запис файлів у вимірювання не входять. Перший прогін виконується для «прогріву» JIT-компілятора і в статистику не потрапляє. Для кожної кількості потоків обробка повторюється

11 разів, у таблицю береться медіана – вона менш чутлива до випадкових затримок операційної системи, ніж середнє. Програму зібрано в конфігурації Release. Повний код наведено в додатку А.

### 3.5 Результат накладання фільтра

Вхідне зображення наведено на рисунку 3.1, згенерований фрактальний шум Перліна – на рисунку 3.2, результат накладання шуму – на рисунку 3.3.

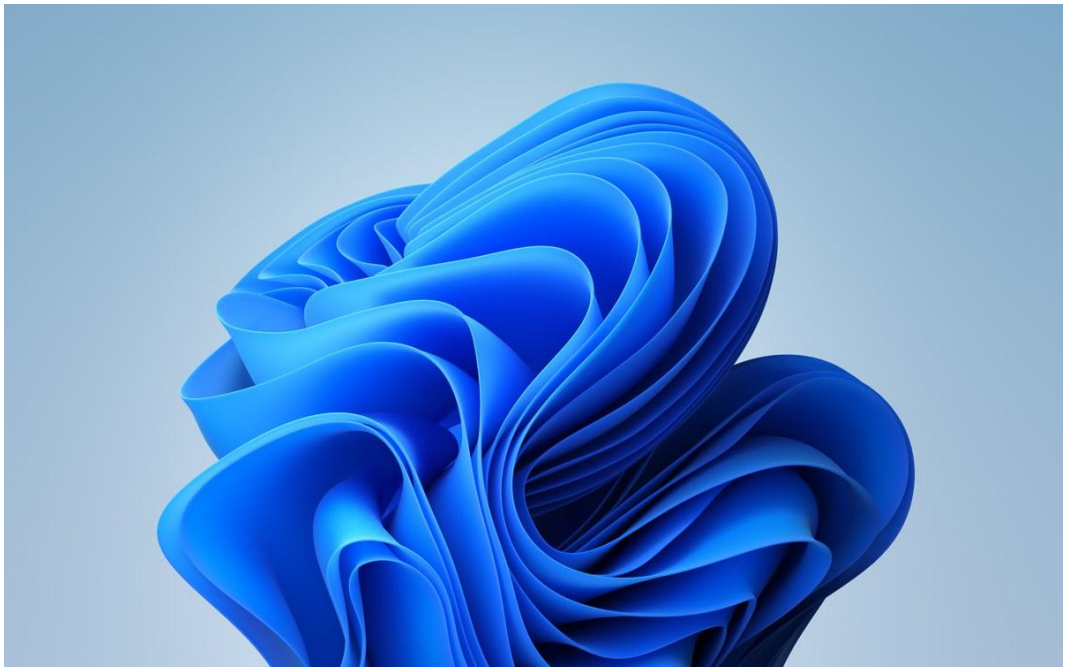


Рисунок 3.1 – Вхідне зображення (3840 × 2400)

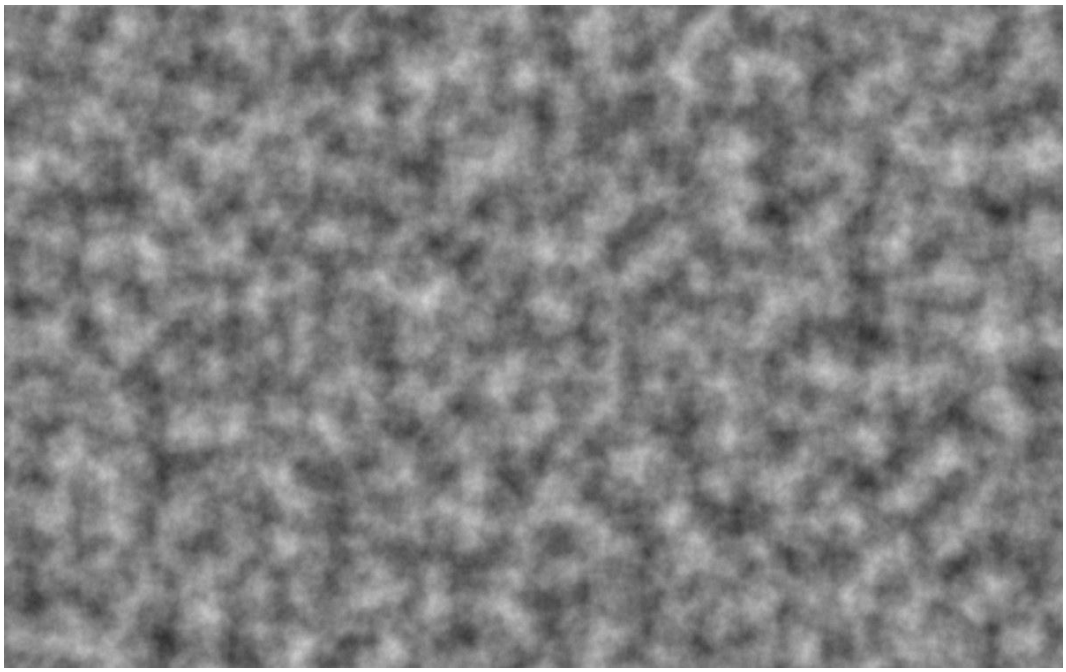


Рисунок 3.2 – Фрактальний шум Перліна (6 октав, масштаб 1/160)

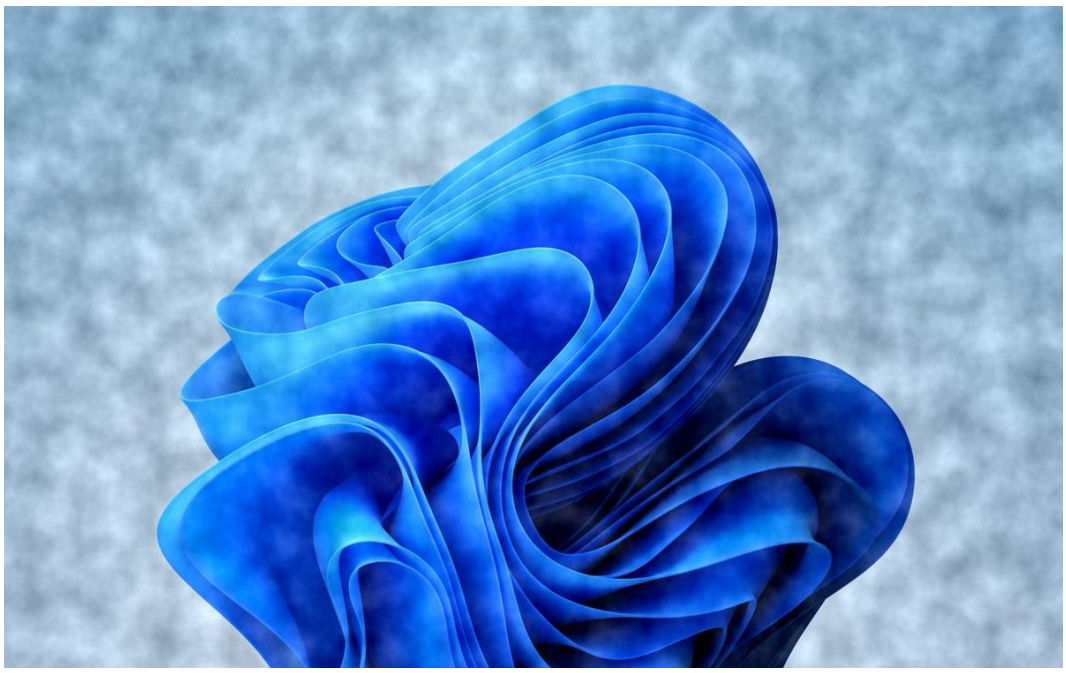


Рисунок 3.3 – Результат накладання шуму Перліна ($k = 70$)

### 3.6 Експериментальні результати

Вимірювання виконано на комп’ютері з процесором AMD Ryzen 5 5600 (6 ядер, 12 потоків), ОС Windows 11 Pro, .NET 10. Крім кількостей потоків із завдання (1, 2, 4, 6, 8, 10, 12, 14, 16), виміряно також 3, 5 та 7 потоків, щоб отримати повний ряд 1–8 для графіка. Результати наведено в таблиці 3.2 та на рисунках 3.4, 3.5. Для всіх кількостей потоків результат побайтово збігся з результатом в одному потоці.

Таблиця 3.2 – Час обробки зображення, прискорення та ефективність

| $p$ | Медіана $T_p$, мс | Мін., мс | Макс., мс | $S_p$ | $E_p$ |
|---|---|---|---|---|---|
| 1 | 1081,4 | 1075,4 | 1113,1 | 1,00 | 1,00 |
| 2 | 541,7 | 537,7 | 562,6 | 2,00 | 1,00 |
| 3 | 366,9 | 358,7 | 494,6 | 2,95 | 0,98 |
| 4 | 289,4 | 278,9 | 308,0 | 3,74 | 0,93 |
| 5 | 245,8 | 238,3 | 266,7 | 4,40 | 0,88 |
| 6 | 217,2 | 206,5 | 235,1 | 4,98 | 0,83 |
| 7 | 210,6 | 207,7 | 217,3 | 5,13 | 0,73 |
| 8 | 205,4 | 199,5 | 224,5 | 5,26 | 0,66 |
| 10 | 179,3 | 177,4 | 188,2 | 6,03 | 0,60 |
| 12 | 166,9 | 164,9 | 183,3 | 6,48 | 0,54 |
| 14 | 200,8 | 172,6 | 216,0 | 5,38 | 0,39 |
| 16 | 202,2 | 193,2 | 206,5 | 5,35 | 0,33 |

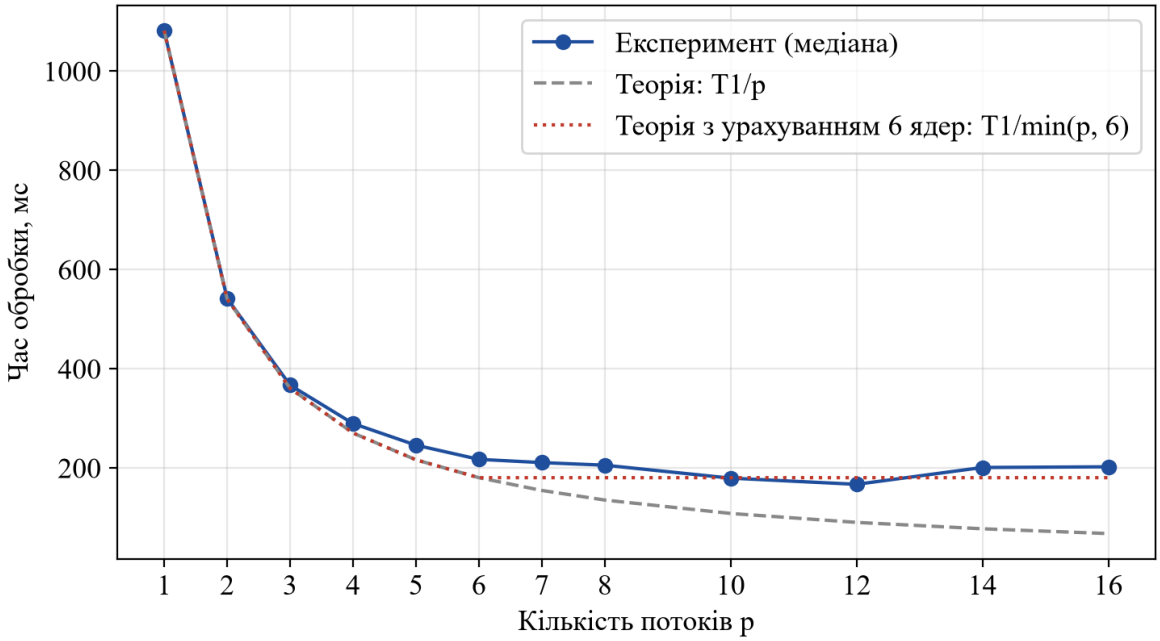



Рисунок 3.4 – Залежність часу обробки зображення від кількості потоків

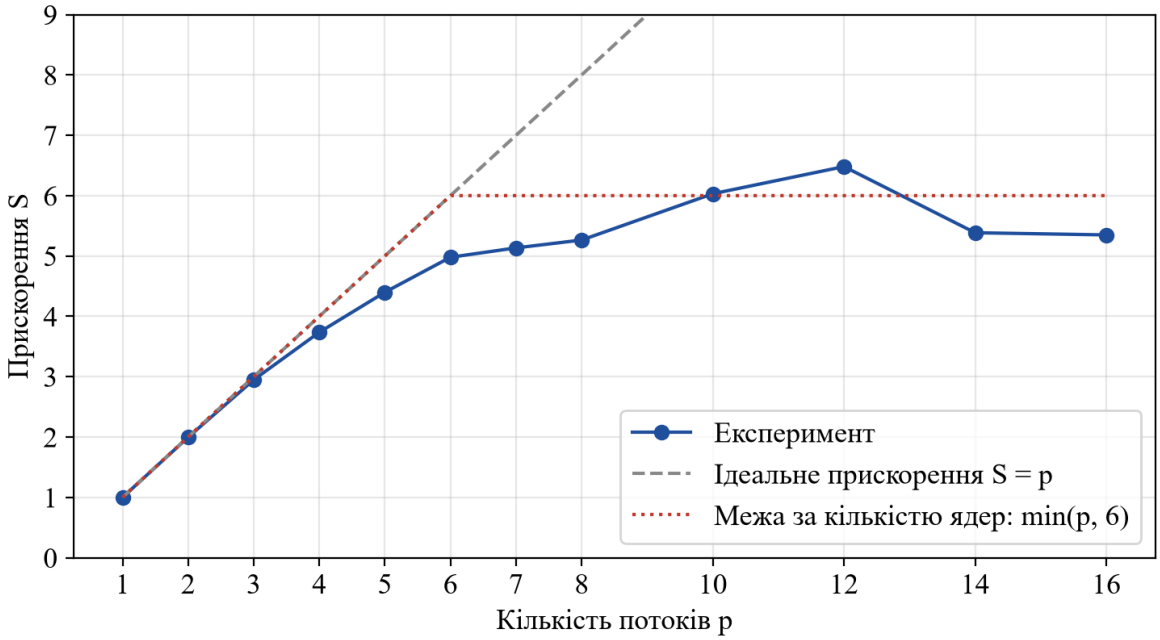



Рисунок 3.5 – Залежність прискорення від кількості потоків

### 3.7 Порівняння з теоретичними результатами

Для 1–3 потоків прискорення практично дорівнює теоретичному: $S_2$ = 2,00, $S_3$ = 2,95 при ефективності понад 0,98 – це підтверджує, що частка послідовного коду та накладні витрати на потоки дуже малі.

Для 4–6 потоків прискорення відстає від min($p$, 6): $S_6$ = 4,98 замість 6 ($E_6$ = 0,83). Ймовірні причини (окремо не вимірювалися): зниження тактової частоти процесора (Precision Boost), коли навантажено всі ядра, а не одне–два; фонові процеси ОС, які займають частину часу ядер; спільна кеш-пам’ять L3 і пропускна здатність пам’яті.

Для 7–12 потоків працює SMT: другий логічний потік на ядрі використовує простої конвеєра, тому прискорення ще зростає і досягає максимуму $S_{12}$ = 6,48 при $p$ = 12, тобто SMT додало близько 30 % до результату 6 потоків. Ефективність при цьому падає до 0,54, бо ефективність рахується на кількість потоків, а фізичних ядер лише 6.

Для 14 та 16 потоків (більше, ніж 12 логічних процесорів) прискорення зменшується до 5,38 та 5,35: частина потоків чекає на процесор, планувальник ОС перемикає контексти, і смуги завершуються нерівномірно – загальний час визначає потік, який отримав процесор останнім. Це узгоджується з теоретичним висновком, що кількість потоків понад кількість логічних процесорів не дає виграшу.

# 4 ВИСНОВКИ

У ході лабораторної роботи розроблено багатопоточний програмний засіб мовою C#, що накладає на зображення фрактальний шум Перліна (варіант 9). Обробка розпаралелена декомпозицією зображення на горизонтальні смуги за кількістю потоків; синхронізація не потрібна, бо потоки записують результат у різні рядки, а коректність підтверджено побайтовим збігом результатів з однопотоковим варіантом.

Теоретично задача майже повністю паралельна ($f \approx 0$), тому на паракомп'ютері $S_p \approx p$, а на 6-ядерному процесорі – $S_p \approx \min(p, 6)$. Експеримент підтвердив лінійне прискорення до 3 потоків ($E > 0{,}98$), показав відставання від теорії на 4–6 потоках через зниження частоти та фонове навантаження і максимальне прискорення 6,48 при 12 потоках завдяки SMT. Подальше збільшення кількості потоків понад кількість логічних процесорів збільшує час обробки.

Отже, оптимальна кількість потоків для такої обчислювальної задачі дорівнює кількості логічних процесорів (12), а за критерієм ефективності – кількості фізичних ядер (до 6).

# ДОДАТОК А

## Код програми

### Файл Perlin.cs

```
namespace PerlinFilter;

/// <summary>
/// Шум Перліна (покращена версія К. Перліна, 2002) та фрактальна сума октав.
/// Клас незмінний після створення, тому його можна безпечно читати з
багатьох потоків.
/// </summary>
public sealed class Perlin
{
    private readonly int[] _p = new int[512]; // таблиця перестановок,
продубльована, щоб не робити & 255 двічі

    public Perlin(int seed)
    {
        var perm = new int[256];
        for (int i = 0; i < 256; i++) perm[i] = i;
        var rnd = new Random(seed);
        for (int i = 255; i > 0; i--) // перемішування Фішера–Єйтса
        {
            int j = rnd.Next(i + 1);
            (perm[i], perm[j]) = (perm[j], perm[i]);
        }
        for (int i = 0; i < 512; i++) _p[i] = perm[i & 255];
    }

    // Згладжувальна крива 6t^5 - 15t^4 + 10t^3: неперервні перша і друга
похідні.
    private static double Fade(double t) => t * t * t * (t * (t * 6 - 15) +
10);

    private static double Lerp(double t, double a, double b) => a + t * (b -
a);

    // Скалярний добуток псевдовипадкового градієнта (одного з 4
діагональних) на вектор відстані (x, y).
    private static double Grad(int hash, double x, double y) => (hash & 3)
switch
    {
        0 => x + y,
        1 => -x + y,
        2 => x - y,
        _ => -x - y,
    };

    /// <summary>Двовимірний шум Перліна, значення приблизно в [-1;
1].</summary>
    public double Noise(double x, double y)
    {
        int xi = (int)Math.Floor(x), yi = (int)Math.Floor(y);
        double xf = x - xi, yf = y - yi;
        xi &= 255; yi &= 255;
```

```
        double u = Fade(xf), v = Fade(yf);

        // Хеші чотирьох кутів клітинки решітки.
        int aa = _p[_p[xi] + yi], ab = _p[_p[xi] + yi + 1];
        int ba = _p[_p[xi + 1] + yi], bb = _p[_p[xi + 1] + yi + 1];

        double x1 = Lerp(u, Grad(aa, xf, yf), Grad(ba, xf - 1, yf));
        double x2 = Lerp(u, Grad(ab, xf, yf - 1), Grad(bb, xf - 1, yf - 1));
        return Lerp(v, x1, x2);
    }

    /// <summary>
    /// Фрактальний шум (fBm): сума октав, у кожній частота ×2, амплітуда
×persistence.
    /// Результат нормовано до [-1; 1].
    /// </summary>
    public double Fractal(double x, double y, int octaves, double
persistence)
    {
        double sum = 0, amp = 1, freq = 1, norm = 0;
        for (int o = 0; o < octaves; o++)
        {
            sum += amp * Noise(x * freq, y * freq);
            norm += amp;
            amp *= persistence;
            freq *= 2;
        }
        return sum / norm;
    }
}
```

## Файл Program.cs

```
using System.Diagnostics;
using System.Drawing;
using System.Drawing.Imaging;
using System.Globalization;
using System.Runtime.InteropServices;
using PerlinFilter;

// ЛР №1 ТВО, варіант 9: накладання шуму Перліна на зображення в кількох
потоках.
// Запуск: dotnet run -c Release -- <вхідне зображення> <тека результатів>
[кількість повторів]

string input = args.Length > 0 ? args[0] : "input.jpg";
string outDir = args.Length > 1 ? args[1] : "results";
int repeats = args.Length > 2 ? int.Parse(args[2]) : 7;
int[] threadCounts = [1, 2, 3, 4, 5, 6, 7, 8, 10, 12, 14, 16];
Directory.CreateDirectory(outDir);

// Параметри фільтра.
const double Scale = 1.0 / 160; // розмір найбільшої «плями» шуму ≈ 160
пікселів
const int Octaves = 6;
const double Persistence = 0.5;
const double Strength = 70; // наскільки сильно шум змінює яскравість
(0..255)
var perlin = new Perlin(seed: 9);

// Читаємо зображення в масив байтів BGR (24 біти на піксель).
using var bmp = new Bitmap(input);
```

```
int width = bmp.Width, height = bmp.Height;
var rect = new Rectangle(0, 0, width, height);
var data = bmp.LockBits(rect, ImageLockMode.ReadOnly,
PixelFormat.Format24bppRgb);
int stride = data.Stride;
byte[] src = new byte[stride * height];
Marshal.Copy(data.Scan0, src, 0, src.Length);
bmp.UnlockBits(data);

Console.WriteLine($"Зображення {width}x{height}, логічних процесорів:
{Environment.ProcessorCount}");

// Обробка рядків [y0; y1). Кожен потік пише лише у свої рядки dst, тому
синхронізація не потрібна.
void ProcessRows(byte[] dst, int y0, int y1)
{
    for (int y = y0; y < y1; y++)
    {
        int row = y * stride;
        for (int x = 0; x < width; x++)
        {
            double n = perlin.Fractal(x * Scale, y * Scale, Octaves,
Persistence);
            int delta = (int)Math.Round(Strength * n);
            int i = row + x * 3;
            dst[i] = Clamp(src[i] + delta);
            dst[i + 1] = Clamp(src[i + 1] + delta);
            dst[i + 2] = Clamp(src[i + 2] + delta);
        }
    }
}

static byte Clamp(int v) => (byte)(v < 0 ? 0 : v > 255 ? 255 : v);

// Зображення ділиться на p горизонтальних смуг однакової висоти, по одній на
потік.
double RunWithThreads(byte[] dst, int p)
{
    var sw = Stopwatch.StartNew();
    var threads = new Thread[p];
    for (int t = 0; t < p; t++)
    {
        int y0 = height * t / p, y1 = height * (t + 1) / p;
        threads[t] = new Thread(() => ProcessRows(dst, y0, y1));
        threads[t].Start();
    }
    foreach (var th in threads) th.Join();
    sw.Stop();
    return sw.Elapsed.TotalMilliseconds;
}

// Еталон: один потік. Перший прогін також «прогріває» JIT.
byte[] reference = new byte[src.Length];
RunWithThreads(reference, 1);

var lines = new List<string> {
"threads;median_ms;min_ms;max_ms;speedup;efficiency;equal_to_1_thread" };
double t1 = 0;
foreach (int p in threadCounts)
{
    var times = new double[repeats];
```

```
    byte[] dst = new byte[src.Length];
    for (int r = 0; r < repeats; r++) times[r] = RunWithThreads(dst, p);
    Array.Sort(times);
    double median = times[repeats / 2];
    if (p == 1) t1 = median;
    bool same = dst.AsSpan().SequenceEqual(reference);
    double s = t1 / median, e = s / p;
    Console.WriteLine($"p={p,2}: медіана {median,8:F1} мс, S={s:F2},
E={e:F2}, збіг з 1 потоком: {same}");
    lines.Add(string.Join(';', p, F(median), F(times[0]), F(times[^1]), F(s),
F(e), same));
}
File.WriteAllLines(Path.Combine(outDir, "results.csv"), lines);

static string F(double v) => v.ToString("F3", CultureInfo.InvariantCulture);

// Зберігаємо результат фільтра і окремо сам шум (для звіту).
SaveBgr(reference, Path.Combine(outDir, "output.jpg"));
byte[] noiseOnly = new byte[src.Length];
for (int y = 0; y < height; y++)
    for (int x = 0; x < width; x++)
    {
        byte g = Clamp((int)Math.Round(127.5 + 127.5 * perlin.Fractal(x *
Scale, y * Scale, Octaves, Persistence)));
        int i = y * stride + x * 3;
        noiseOnly[i] = noiseOnly[i + 1] = noiseOnly[i + 2] = g;
    }
SaveBgr(noiseOnly, Path.Combine(outDir, "noise.jpg"));
Console.WriteLine($"Результати збережено в {Path.GetFullPath(outDir)}");

void SaveBgr(byte[] pixels, string path)
{
    using var img = new Bitmap(width, height, PixelFormat.Format24bppRgb);
    var d = img.LockBits(rect, ImageLockMode.WriteOnly,
PixelFormat.Format24bppRgb);
    Marshal.Copy(pixels, 0, d.Scan0, pixels.Length);
    img.UnlockBits(d);
    img.Save(path, ImageFormat.Jpeg);
}
```